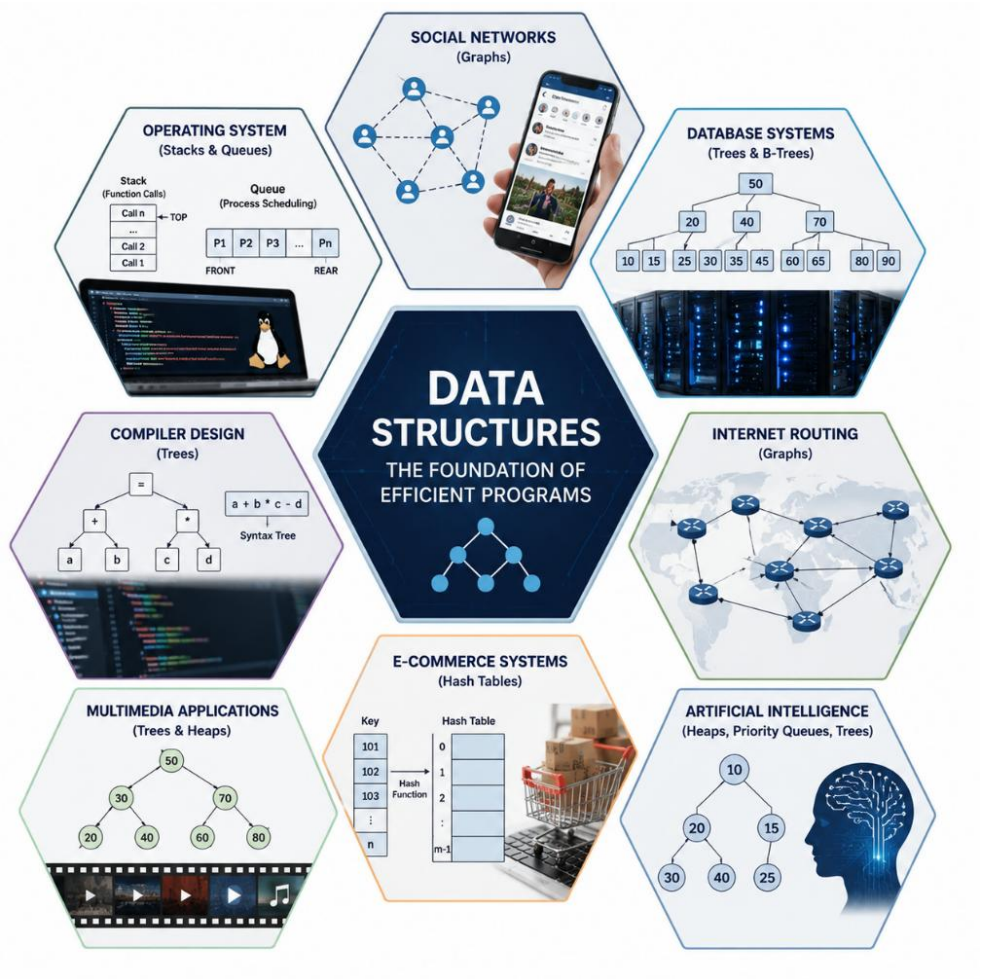
Data Structure & Algorithm Lab

ECE2132



Department of Electronics & Communication Engineering

By:

Dr. D.V. Prashant
Dr. Kamal Kishore Upadhyay

Manipal University Jaipur,

Dehmi Kalan, Off Jaipur-Ajmer Expressway, Jaipur, (Raj.)

Rajasthan 303007

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

(AY 2026-27)

Experiment No. 12

Write a Program to Implement circular Linked List with Insertion, Deletion, and Traversal Operations.

AIM: To implement a circular linked list in C++ and perform insertion, deletion, and traversal operations.

SOFTWARE USED: Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

THEORY: A circular linked list is a type of linked list in which the last node points back to the first node instead of pointing to NULL. Thus, the nodes form a circle.

Each node contains two parts:

• Data: Stores the value of the node.

• Next: Stores the address of the next node.

Traversal can continue around the list and eventually return to the starting node, so a suitable stopping condition is required to avoid an infinite loop.

Types of Circular Linked Lists

1. Circular Singly Linked List: Each node points to the next node, and the last node points to the first node.

2. Circular Doubly Linked List: Each node contains pointers to the next and previous nodes, forming a circular structure in both directions.

Basic Operations

Insertion: Adds a new node. This program inserts at the end.

Deletion: Removes a node. This program deletes from the beginning.

Traversal: Displays each node's data, stopping when traversal returns to the starting node.

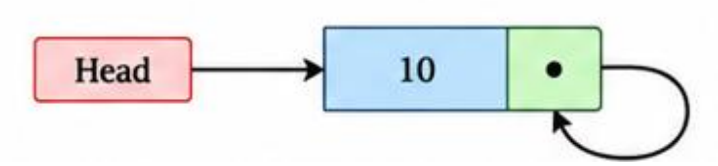


Figure.1: Circular linked list with one node

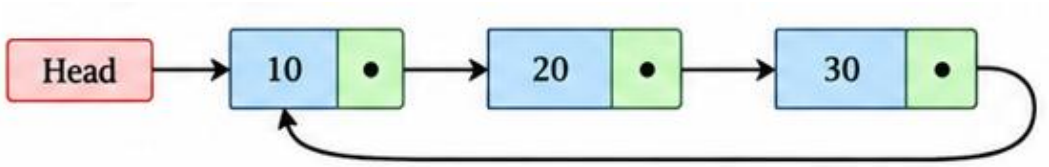


Figure.2: List before insertion.

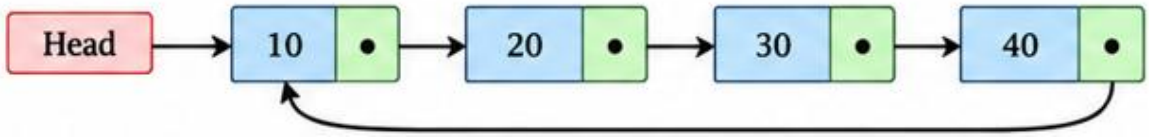


Figure.3: List after insertion.

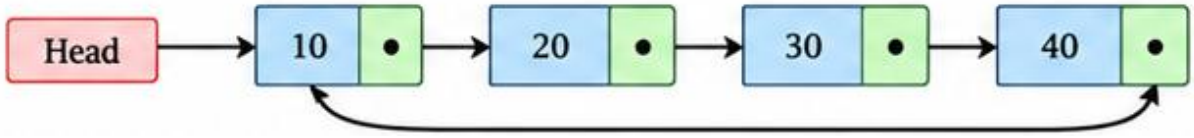


Figure. 5: List before deletion.

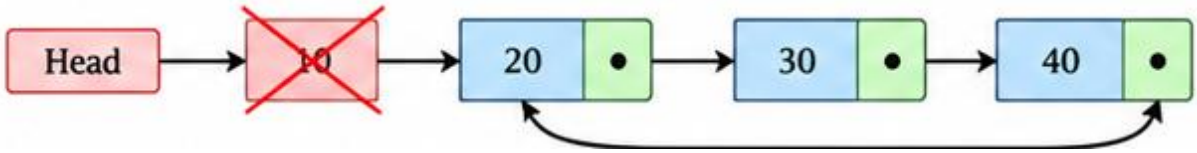


Figure. 6: List after deletion.

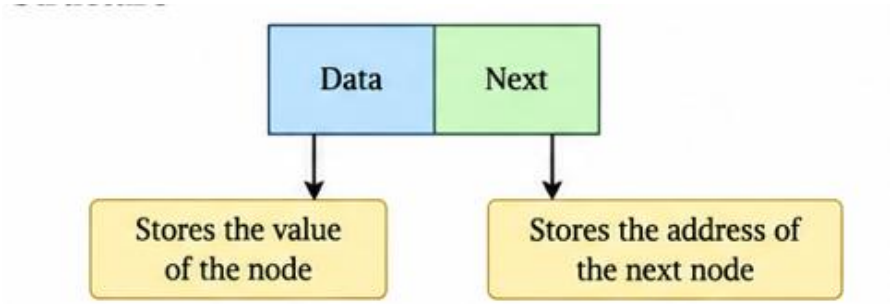


Figure. 7: Structure of node.

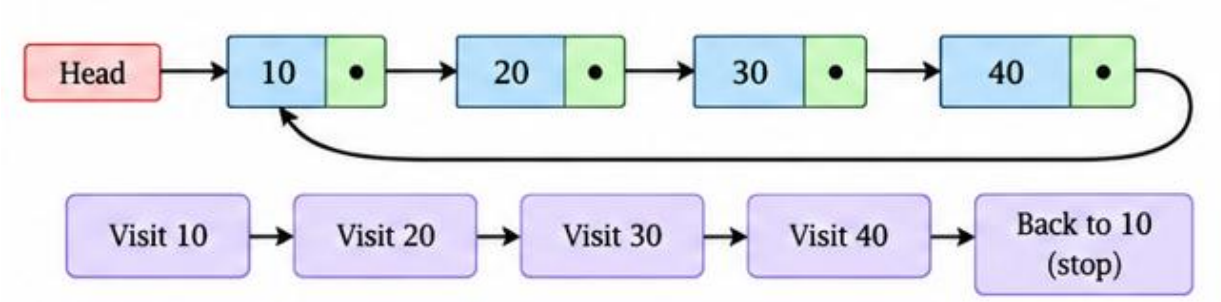


Figure. 8: Traversal of Circular linked-list.

Feature	Singly Linked List	Circular Linked List	Doubly Linked List
Last node pointer	NULL	Points to head	NULL
Traversal	One way	One way (circular)	Both ways
Memory per node	1 pointer	1 pointer	2 pointers
End condition for traversal	temp == NULL	temp == head	temp == NULL
Applications	General purpose	Round-robin scheduling, cyclic task management	Bidirectional traversal

Fig.9 Comparison table

ALOGRITHM:

Insertion at the End

1. Create a new node and store the given value.
2. If the list is empty, make the new node point to itself and set it as head.
3. Otherwise, traverse until reaching the last node.
4. Set the new node's next pointer to head.
5. Set the last node's next pointer to the new node.

Deletion from the Beginning

1. Check whether the list is empty.
2. If only one node exists, delete it and set head = NULL.
3. Otherwise, find the last node.
4. Move head to the next node.
5. Update the last node's next pointer to the new head.
6. Delete the previous first node.

Traversal

1. Check whether the list is empty.
2. Start from head.
3. Display the current node's data.
4. Move to the next node.
5. Stop when the pointer returns to head.

PROGRAM (C++):

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node* next;
};

Node* head = NULL;

void insert(int value)
{
    Node* newNode = new Node;
    newNode->data = value;

    if(head == NULL)
    {
        head = newNode;
        newNode->next = head;
    }
    else
    {   Node* temp = head;

        while(temp->next != head)
        {
            temp = temp->next;
        }
        temp->next = newNode;
    }   newNode->next = head;
    cout << value << " inserted into list." << endl;
}

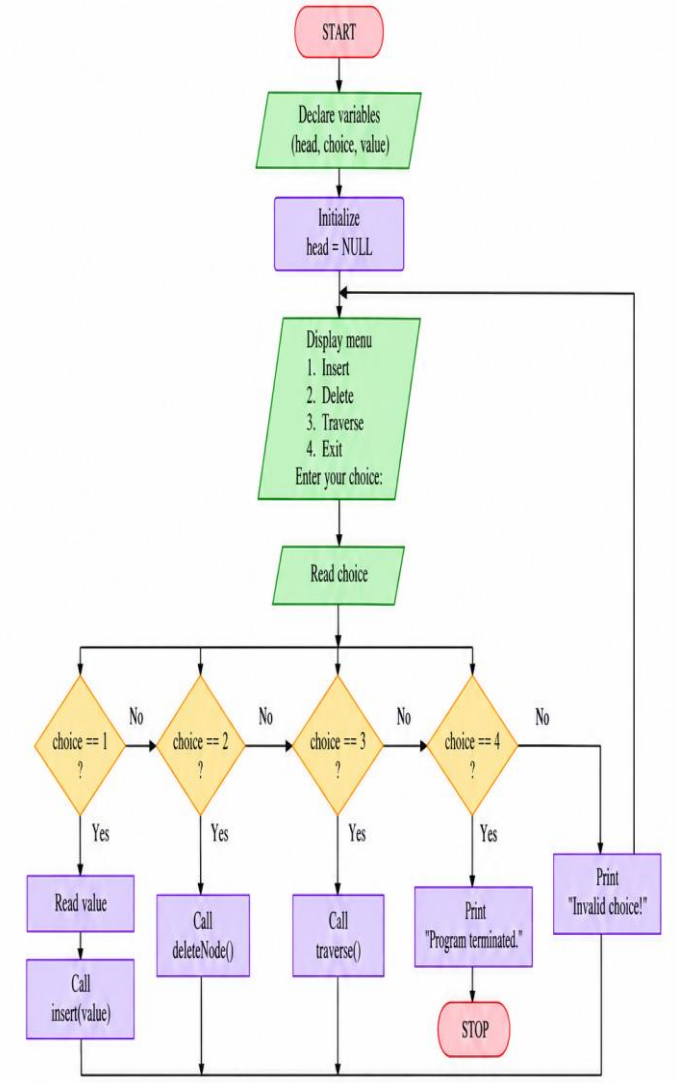
void deleteNode()
{
    if(head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else if(head->next == head)
    {   cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
    }
    else
    {   Node* temp = head;
        Node* last = head;

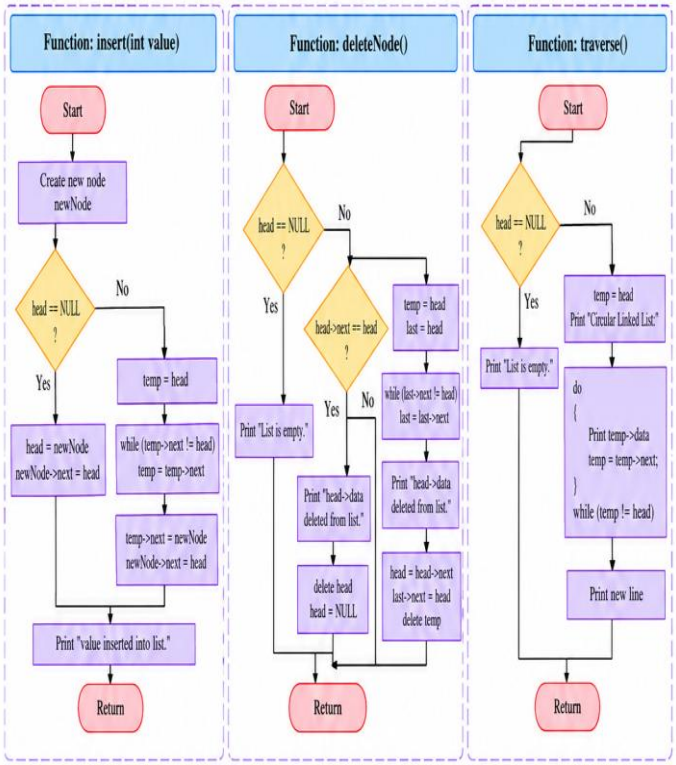
        while(last->next != head)
        {
            last = last->next;
        }
        cout << head->data << " deleted from list." << endl;
        head = head->next;
        last->next = head;
    }   delete temp;
}

void traverse()
{   if(head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {   Node* temp = head;
        cout << "Circular Linked List: ";
        do
        {   cout << temp->data << "  ";
            temp = temp->next;
        }
        while(temp != head);
    }
}
int main()
{   int choice, value;

    do
    {   cout << "\n1. Insert
        cout << "\n2. Delete
        cout << "\n3. Traverse
        cout << "\n4. Exit
        cout << "Enter your choice: ";
        cin >> choice;

        if(choice == 1)
```





EXPLANATION OF THE PROGRAM:

Operation	Description	Time Complexity
Insertion	Adds a node at the end	O(n)
Deletion	Removes the first node; finds the last node to reconnect the circle	O(n)
Traversal	Displays all nodes	O(n)

SAMPLE INPUT 1:

1. Insert

2. Delete

3. Traverse

4. Exit

Enter your choice: 1

Enter value to insert: 10

10 inserted into list.

Enter your choice: 1

Enter value to insert: 20

20 inserted into list.

Enter your choice: 1

Enter value to insert: 30

30 inserted into list.

Enter your choice: 3

Circular Linked List: 10 20 30

Enter your choice: 2

10 deleted from list.

Enter your choice: 3

Circular Linked List: 20 30

Enter your choice: 4

Program terminated.

PERFORMANCE ANALYSIS

The performance of a circular linked list depends on the operation being performed.

Insertion Operation

In this program, insertion is performed at the end. The program traverses the list to find the last node before inserting the new node.

Time Complexity = O(n)

Deletion Operation

Deletion is performed from the beginning. The program finds the last node to update its pointer to the new head before deleting the first node.

Time Complexity = O(n)

Traversal Operation

Traversal visits each node in the circular linked list until the pointer returns to the starting node (head).

Time Complexity = O(n)

TIME COMPLEXITY

Operation	Time Complexity
Insertion at end	O(n)
Deletion at beginning	O(n)
Traversal	O(n)

SPACE COMPLEXITY

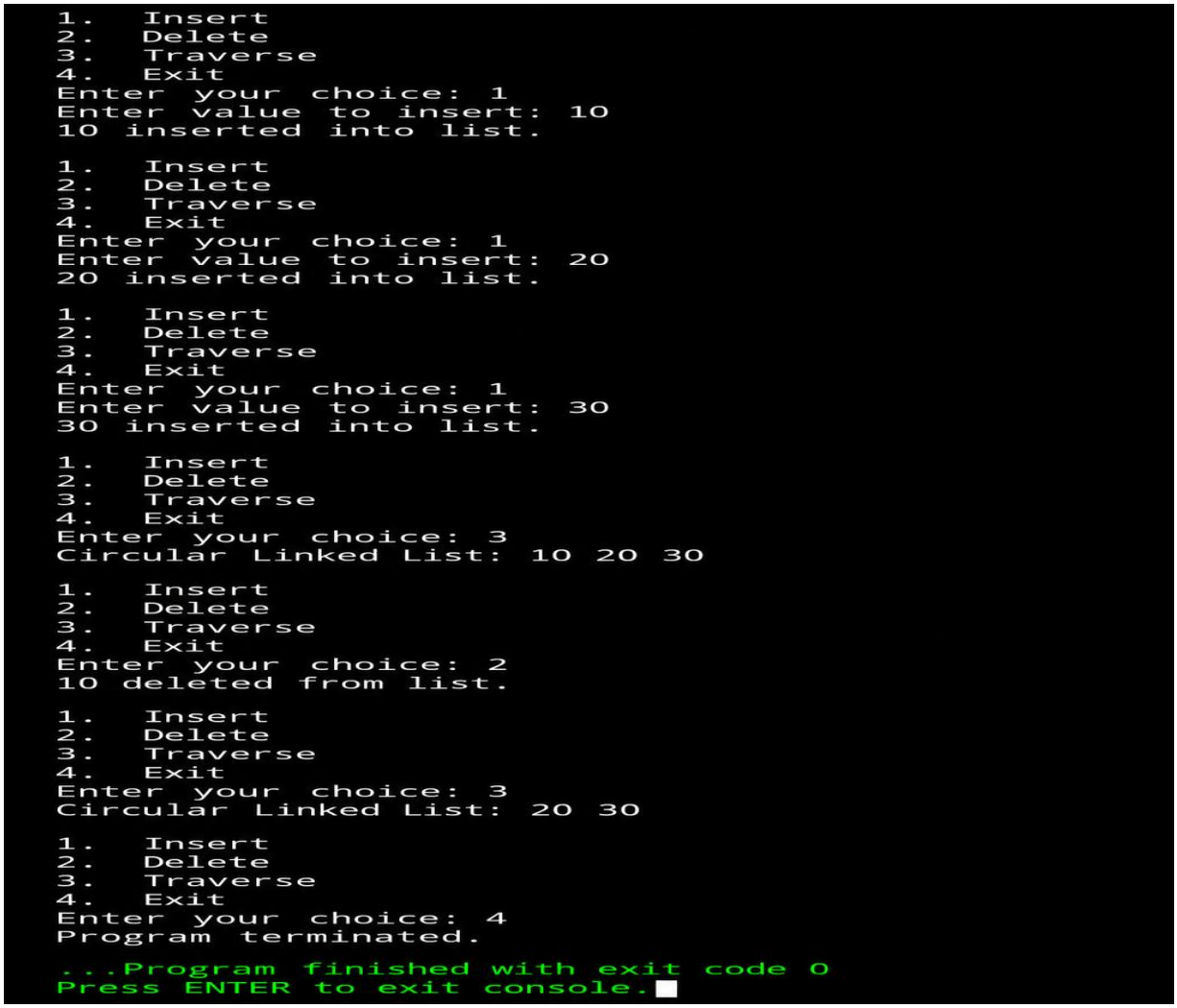
A separate node is created for every element inserted into the circular linked list. Each node stores data and a pointer to the next node. The list requires additional space proportional to the number of nodes.

Space Complexity = O(n)

The insertion, deletion, and traversal functions use only a constant number of additional pointer variables, so their auxiliary space complexity is O(1), excluding the memory occupied by the linked-list nodes.

IMPLEMENTATION OF CODE

```cpp
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node* next;
};
Node* head = NULL;
void insert(int value)
{
    Node* newNode = new Node;
    newNode->data = value;
    if(head == NULL)
    {
        head = newNode;
        newNode->next = head;
    }
    else
    {
        Node* temp = head;
        while(temp->next != head)
        {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
    }
    cout << value << " inserted into list." << endl;
}
void deleteNode()
{
    if(head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else if(head->next == head)
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
    }
    else
    {
        Node* temp = head;
        Node* last = head;
        while(last->next != head)
        {
            last = last->next;
        }
        cout << head->data << " deleted from list." << endl;
        head = head->next;
        last->next = head;
        delete temp;
    }
}
void traverse()
{
    if(head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {   Node* temp = head;
        cout << "Circular Linked List: ";
        do
        {
            cout << temp->data << " ";
            temp = temp->next;
        }
        while(temp != head);
    }   cout << endl;
}
int main()
{
    int choice, value;
    do
    {   cout << "\n1. Insert" << endl;
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;
        if(choice == 1)
        {   cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if(choice == 2)
        {   deleteNode();
        }
        else if(choice == 3)
        {   traverse();
        }
        else if(choice == 4)
        }   cout << "Program terminated." << endl;
        else
        {   cout << "Invalid choice!" << endl;
    } while(choice != 4);
}   return 0;
```



APPLICATIONS: Round-robin CPU scheduling, Multiplayer turn management, Circular queues, Repeated playlist navigation, Cyclic task scheduling.

RESULT:

Thus, a circular linked list was implemented successfully in C++, and insertion, deletion, and traversal operations were performed.

ASSIGNMENT:

- WAP to insert a node at the beginning of a circular linked list.
- WAP to insert a node at the end of a circular linked list.
- WAP to delete a node from the beginning of a circular linked list.
- WAP to delete a node from the end of a circular linked list.
- WAP to search for an element in a circular linked list.
- WAP to count the total number of nodes in a circular linked list.
- WAP to display the elements of a circular linked list in reverse order.
- WAP to insert a node at a specified position in a circular linked list.
- WAP to delete a node at a specified position in a circular linked list.